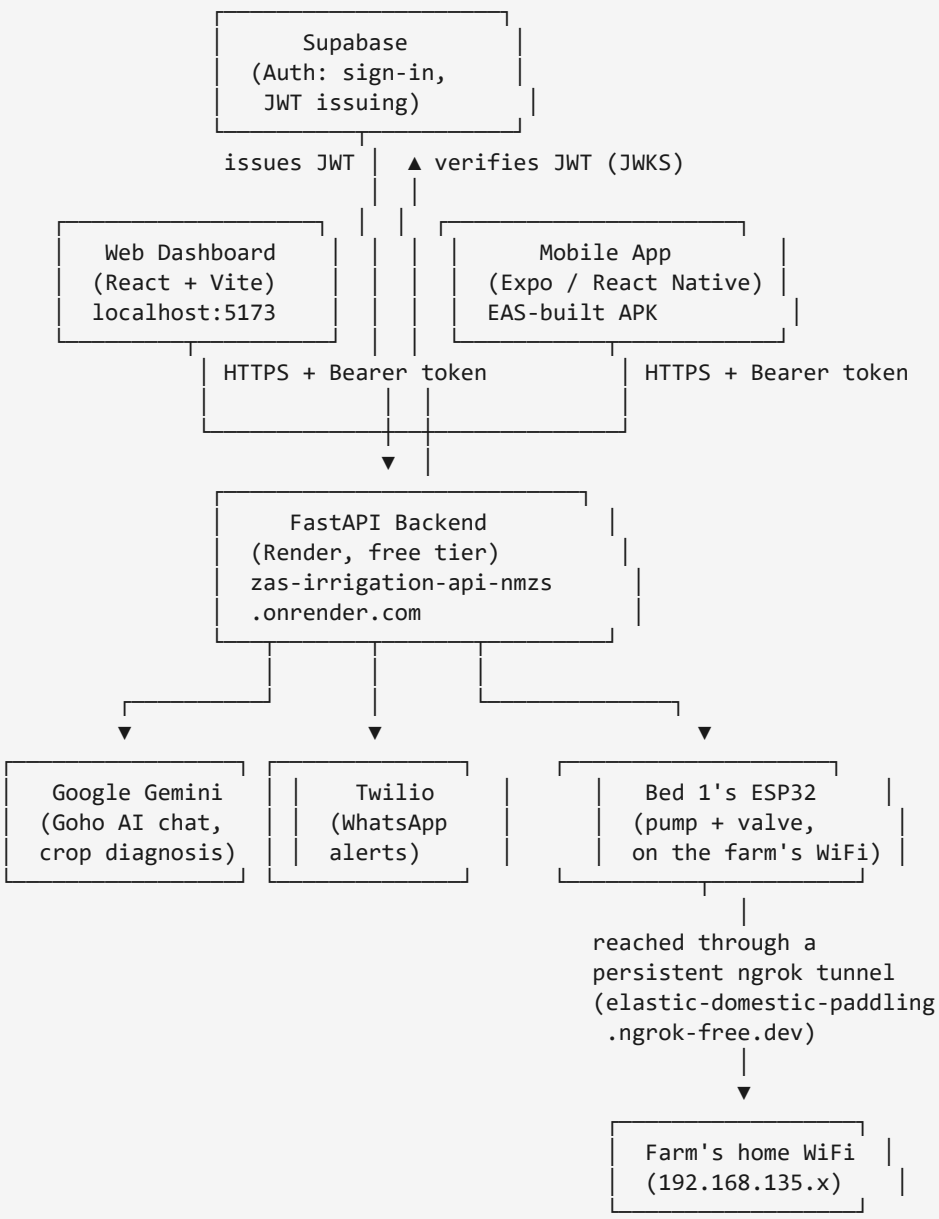
System Architecture — Agri-Intelligence 2026 Smart Irrigation Project

Generated 2026-09-09. Companion to `SOFTWARE_USED.txt` (what each piece is built with) and `CHANGED_FILES.txt` (running history of how it got here).

---

1. Overview

Two client apps (a web dashboard and a mobile app) share one backend API and one identity provider. The backend is the only thing that talks to Gemini, Twilio, and the farm's physical hardware — neither client ever calls those directly. Only **Bed 1** has real hardware behind it (an ESP32 driving a pump + valve); Beds 2/3 are simulated in the backend's in-memory state so the rest of the system behaves identically either way.



---

2. Components

2.1 Web Dashboard (`irrigation-dashboard/`)

React + Vite single-page app. Talks to the backend through one shared client (`src/lib/apiClient.js`) that attaches the current Supabase session's bearer token to every request and centrally handles a 401 (expired/invalid session) by signing the user out. Runs locally during development (`localhost:5173` → backend on `localhost:8021`); a production deploy would point `VITE_API_URL` at the Render URL instead.

2.2 Mobile App (`mobile-app/`)

Expo/React Native app, same backend contract as the dashboard (`src/services/api.ts`), same Supabase auth. Ships two ways: - **Dev**: Metro/Expo dev server, `EXPO_PUBLIC_API_URL` from `.env` (defaults to `localhost:8021`), or a runtime override set from the in-app Settings screen (persisted on-device — lets a phone be pointed at a LAN IP, tunnel, or the deployed URL without a rebuild). - **Production**: an EAS-built APK with `EXPO_PUBLIC_API_URL` baked in at build time (`eas.json`'s `preview`/`production` profiles) pointing at the deployed Render backend — this is what the farmer installs and just opens, no configuration needed.

2.3 Backend API (`backend/`)

FastAPI app, the single source of truth for farm state (bed moisture, irrigation status, alerts, activity log — currently in-memory, reset on restart) and the only component with credentials for every external service. Every `/api/*` route requires a valid Supabase-issued bearer token, verified against Supabase's public JWKS via PyJWT — no shared secret between backend and clients, just the same public key Supabase itself publishes.

Runs two ways: - **Local dev**: `uvicorn main:app --port 8021`, reachable only on `localhost` / the LAN. - **Deployed**: Render web service (`render.yaml`), free tier, auto-deploys from the `main` branch on GitHub. Publicly reachable over HTTPS, which is what makes the app usable off the farm's WiFi at all.

2.4 Authentication (Supabase)

Shared identity provider for both clients — one account, works in either app. Supabase issues short-lived JWTs on sign-in; the backend never talks to Supabase's auth API directly for verification, it just checks each token's signature against the project's JWKS (`SUPABASE_URL` env var), so verification works even if Supabase itself is briefly unreachable.

2.5 AI Services (Google Gemini)

Two features, one SDK (`google-genai`), backend-only (the API key never reaches a client): the **Goho AI** conversational assistant, and photo-based **crop-health diagnosis**.

2.6 Alerts (Twilio)

Backend sends WhatsApp messages for critical events (e.g. a bed's moisture dropping below its trigger threshold) via Twilio's WhatsApp Business API. Best-effort — a failed WhatsApp send is logged, not raised as an error, so it never blocks the irrigation action that triggered it.

2.7 Hardware Layer (Bed 1's ESP32)

`firmware/sensor-node/bed1-manual-irrigation/` — a small HTTP server on the ESP32 itself (`/irrigate?sec=N&key=...`, `/stop?key=...`, `/status?key=...`), driving a pump (via a 2N2222-piloted automotive relay) and a solenoid valve (via a relay module) through a timed open-valve → pump → pump-off → close- valve sequence. Every endpoint requires a shared-secret `key` query param (`ESP32_SHARED_SECRET`, matched on both the firmware and the backend) — this board has no other authentication, so once it's reachable off the LAN this key is the only thing stopping an outsider from running the pump.

The ESP32 itself only ever accepts inbound connections on its home WiFi; it never dials out. **A persistent ngrok tunnel**, run on a machine on that same WiFi, is what makes it reachable from the Render-hosted backend at all — without it, the deployed backend can only simulate Bed 1 like the other beds. Only Bed 1 has physical hardware; Beds 2 and 3 are backend-simulated.

---

3. Request Flows

3.1 Sign-in

1. Client (web or mobile) calls Supabase directly with email/password.
2. Supabase returns a session (access token + refresh token).
3. Client stores the session (AsyncStorage on mobile, Supabase's own storage on web) and attaches `Authorization: Bearer <token>` to every backend call from then on.

3.2 Loading the dashboard

1. Client calls `GET /api/dashboard` with its bearer token.
2. Backend verifies the token's signature via Supabase's JWKS.
3. Backend returns current bed states, system info, and activity log from its in-memory store.
4. Client renders it; mobile additionally caches it to AsyncStorage so a later offline load still shows the last-known data with an "Offline" banner.

3.3 Triggering irrigation on Bed 1 (the real-hardware path)

1. Client calls `POST /api/irrigation/start/1` with a bearer token and a requested duration.
2. Backend verifies the token, then — **before** touching any in-memory state — calls the ESP32's `/irrigate?sec=N&key=...` through the ngrok tunnel.
3. If the ESP32 doesn't respond (tunnel down, board offline, wrong key), the backend returns `502` and leaves the bed's state untouched, so the app never shows "irrigating" while nothing is actually happening in the field.
4. If it succeeds, the backend flips the bed to `irrigating: true` and returns it. The ESP32 itself now owns the timing — it runs the full valve → pump → pump-off → valve sequence on its own hard-coded timer, independent of whether the client stays connected.
5. Triggering Beds 2/3 skips step 2 entirely — the backend just flips its own mock state and applies a moisture bump on `/stop`, since there's no real hardware to call.

3.4 AI crop diagnosis

1. Client uploads a photo to `POST /api/crops/diagnose` (bearer token).
2. Backend forwards the image to Gemini with a diagnosis prompt.
3. Gemini's structured response is relayed back to the client as-is — the backend doesn't store or reprocess the image.

3.5 Critical alert

1. Some backend action (e.g. a moisture reading crossing a bed's trigger threshold) calls the Twilio helper.
2. Twilio sends a WhatsApp message to the configured farmer number.
3. Failure here is logged and swallowed, never surfaced to the client as an error — an alert not sending shouldn't block irrigation from starting.

---

4. Deployment Topology

Piece	Where it runs	Reachable from
Backend API	Render (free tier), auto-deploys from `main` on GitHub	Public internet (HTTPS)
Web Dashboard	Local dev machine (`npm run dev`)	`localhost` only, today
Mobile App (dev)	Expo dev server on a dev machine	Same LAN, or via `expo start --tunnel` for remote testing
Mobile App (production)	EAS-built APK, installed on the farmer's phone	Anywhere — talks to the deployed Render URL directly
Bed 1's ESP32	Physically at the farm, on its home WiFi	LAN directly, or publicly via the ngrok tunnel
ngrok tunnel client	A machine on the farm's WiFi (must stay on/connected for remote hardware control to work)	Bridges the ESP32's LAN to the public ngrok domain
Supabase	Supabase's own cloud	Public internet — both clients and the backend talk to it

Known fragility: remote *hardware* control (Bed 1's real pump) depends on whatever machine is running the ngrok tunnel staying powered on and connected — if it goes offline, remote data/dashboard access keeps working fine (that's served entirely by Render + Supabase), but the deployed backend falls back to `502`s on Bed 1's trigger endpoint until the tunnel machine is back. A small always-on device (e.g. a Raspberry Pi) at the farm would remove this single point of failure; today it's whatever laptop happens to be running it.

---

5. Security Notes

- **Client → Backend**: every `/api/*` route requires a valid Supabase JWT, verified against Supabase's public JWKS (no shared secret to leak).
- **Backend → ESP32**: gated by `ESP32_SHARED_SECRET`, a static shared string (not a rotating token) — adequate for a single-board hobby/demo deployment, not a substitute for real device-level auth at production IoT scale.

- **CORS**: the backend's allowed-origins list covers the known web dashboard origins; native app traffic (mobile) sends no `Origin` header and is unaffected by CORS either way.
- **Secrets**: all third-party API keys (Gemini, Twilio, the ESP32 shared secret) live only in `backend/.env` locally and as Render environment variables in production — never shipped inside either client app.